

型付き AI 記述言語 AIPL の設計と実装

Yasushi Kodama

kodamay@

2026-05-13

Abstract

本稿はアクター指向の並列・並行言語 **AIPL** (Actor-based Intelligent Parallel Language; 旧称 ABCL/c+) の設計と実装を、一つの言語に対する **7 種のランタイム実装** と **8 種のコード生成ターゲット** を横断する形で記述する。本研究の貢献は四つある。第一に、1986 年の ABCL/1 系譜のアクターモデルに現代的な LLM 連携を組み込み、`ai_call(provider, prompt)` プリミティブとして第一級化した。第二に、Hindley-Milner 推論を中心に置きつつ、段階的型注釈・フロー敏感推論・実行時セッション型を選択的に重ねた**多層型システム**を、異なる実装ごとに切り替え可能な独立した次元として整理した。第三に、同一の `.abcl` ソースから OS スレッド (1:1) / 軽量スレッド (M:N) / 協調イベントループの 3 並行モデルすべてに展開できる**ターゲット非依存セマンティクス**を確立した。第四に、AIPL を AIPL で記述する完全な**セルフホスト塔** (Level A-C-3, 37/37 スモークテスト合格) を構築し、言語仕様の自己一貫性を実証した。全 7 ランタイム × 全機能を 91 ページの比較表に集約した結果、言語表面 (`send/now/future/await`) は実装間で同型に保たれる一方、スケーラビリティ・型保証・AI 統合の各軸では各実装がそれぞれ異なる象限を覆っていることが明らかになった。

Contents

1	はじめに	1
2	背景	2
2.1	アクターモデルと ABCL/1 系譜	2
2.2	現代の型システムの蓄積	2
2.3	LLM 連携の必要性	2
3	言語設計	2
3.1	コア構文	2
3.2	多層型システム	3
3.3	LLM 統合	4
3.4	3 並行モデルの統一	4
4	実装	4
4.1	7 ランタイム	5
4.2	8 コード生成ターゲット	5
4.3	コード生成の中核技術	5
5	セルフホスト塔	6
6	評価	7
6.1	サンプル網羅性	7
6.2	スモークテスト結果 (2026-05-13 最新)	7
6.3	型推論のクロス検証	7
6.4	LLM 統合の実証	7
6.5	WebSocket 統合	8

7 関連研究	8
8 結論と今後の課題	8

1 はじめに

並行計算と人工知能の境界はかつてないほど混ざりつつある。大規模言語モデル (LLM) を中心とする AI システムは、本質的に複数のエージェントが協調するアーキテクチャに収束しつつあり、これは 1980 年代の Hewitt のアクターモデル [1] と Yonezawa らの ABCL/1 [2] が描いた未来像と驚くほど近い。しかし当時の言語設計には LLM や型推論・線形性・セッション型など、その後 30 年の研究成果が当然ながら含まれていなかった。

本研究は、アクター言語の系譜を母艦としつつ現代の言語理論の成果を重ねる試みとして **AIPL** (Actor-based Intelligent Parallel Language) を提案する。AIPL は次の四つを同時に達成することを目標とする:

1. **アクター意味論の保存**: `send/now/future/await` の四プリミティブで Hewitt 型アクターを記述できる。
2. **多層型システム**: Hindley-Milner 推論を基底に、段階的注釈・フロー敏感型・効果型・線形型・所有型・セッション型を選択的に重ねられる。
3. **LLM 第一級統合**: `ai_call` を組み込みとし、プロバイダ (Gemini / Claude / GPT) を整数 1 / 2 / 3 で指定する。トークン予算・同時実行上限・フォールバック連鎖をランタイムが管理する。
4. **実装多様性**: 同一の `.abcl` ソースを Python / OCaml / JS / C / Pony / Erlang / Go / SWI-Prolog の各実装で実行可能とする。

論文の構成は次の通りである。§2 で背景を、§3 で言語設計を、§4 で 7 ランタイム + 8 コード生成ターゲットの実装を述べる。§5 でセルフホスト塔、§6 で評価結果、§7 で関連研究、§8 で結論と今後の課題を示す。

2 背景

2.1 アクターモデルと ABCL/1 系譜

Hewitt らの提案したアクターモデル [1] は、計算を **独立に状態を持つアクター間の非同期メッセージ送信**として定式化する。AIPL の直接の先祖である ABCL/1 [2] は、これに次の三種類の送信を導入した:

- *past type* (`send`) — fire-and-forget の非同期送信。
- *now type* (`now`) — 同期送信, 呼び出し側を返答到着までブロック。
- *future type* (`future`) — 非同期送信, 後で `await` で値取得。

これら三種は Erlang ($\rightarrow !$ と `gen_server:call`) や Akka ($\rightarrow \text{tell} / \text{ask}$), Pony の `behaviours` など、後続のアクター言語に直接または間接に継承されている。

2.2 現代の型システムの蓄積

2010 年代以降、並行プログラムの安全性を高めるための型理論的成果が複数得られている:

- **capability effects** [4]: 関数の副作用を型レベルで追跡 (`!{fs, ai, net, mut}` 形式)。
- **linear / affine types** [5]: 値の使用回数を型レベルで制限 (`use-after-move` を静的に検出)。
- **ownership / capabilities** [6]: フィールドのアクセス権を型階層に組込む (Rust や Pony で実用化)。

- **session types** [7]: プロトコル全体を一つの型として表現, 通信順序の不正を静的または動的に検出.

AIPL の Phase 11-17 はこれらを順次取り入れる.

2.3 LLM 連携の必要性

LLM 駆動の AI システムは, 複数のエージェント (planner / solver / reviewer など) が非同期に協調する典型例である. これを既存の汎用言語で書くと, スレッド管理・タイムアウト・予算管理・フォールバックといった雑事のコードが本来のドメインロジックを覆い隠してしまう. AIPL は `ai_call` を組み込みとして第一級化し, これらを言語仕様の側で吸収することを試みる.

3 言語設計

3.1 コア構文

AIPL のコア構文を最小限のクラス例で示す:

```
class Counter {
  var count = 0;
  method tick() {
    count = count + 1;
    print("count = " + count);
  }
  method get() {
    reply(count);
  }
}

var c = new Counter();
send c.tick();           // past — fire-and-forget
var v = now c.get();      // now — block, return reply
var f = future c.get();   // future — handle to async reply
var w = await(f);         // ... pick up the future later
```

主な構造的要素:

- **class 宣言.** 各クラスはアクターのテンプレートで, フィールド (`var`) とメソッド (`method`) を持つ.
- **init メソッド**が `new C(args)` で自動的に呼ばれる.
- メッセージ送信は `send` / `now` / `future` の三種.
- `await(f)` で `future` を解決, `reply(v)` で同期送信に応答.
- `become C(args)` でアクターのクラスを実行時に差し替え.
- `select` で複数チャネルからの選択受信 (Phase 13).

3.2 多層型システム

AIPL の型システムは複数の独立した層で構成される. 実装ごとにどの層を有効にするかが異なる (§4 参照).

Phase 11: 段階的型注釈 変数・パラメータ・戻り値に `:` `int` 形式で型注釈を付与する. 注釈の無い場所は `any` 扱いの段階的型付け.

```
class Counter {
  var count: int = 0;
  method init(n: int) { count = n; }
  method get() -> int { reply(count); }
}
```

Phase 12: capability 効果 関数・メソッドのシグネチャに副作用集合を宣言する:

```
method save(path: string) !{fs} { write_file(path, dump()); }
method ask(q: string) !{ai, net} { reply(ai_call(q)); }
```

呼び出し側はこれらの効果集合の和を上位に伝搬する必要がある。

Phase 13: CSP チャンネル `channel[T]` 型と `channel_send` / `channel_rcv` / `select_rcv` ビルトインで CSP [3] 風通信を提供。

Phase 14: 線形型 `linear` 型の値は一度しか消費できない。データベースハンドルやファイル記述子など、二重解放 (double-consume) や 解放後使用 (use-after-move) を静的に検出する。

Phase 15: 所有 (owned/pub) クラスフィールドはデフォルト `private.pub` `var` 宣言したものだけが外部から書き換え可能。アクターのカプセル化を型階層で表現する。

Phase 16: transient cast `any` 境界で実行時型キャストを挿入, `TransientCastError` を発する `gradual typing` の健全性を最低限保証する仕組み。

Phase 17: 構造化並行 `scope { future ... }` ブロック内で起動した `future` は、ブロック終端で自動的に `join` される (Java の Project Loom と類似)。

Phase 11+ 追加: セッション型 (実行時) プロトコル全体を一つの仕様として宣言し、各送受信を実行時に検証する:

```
session_define("Solve",
  "solver.solve(string) ! string"
  " -> reviewer.review(string,string) ! string"
  " -> planner.solve(string) ! string");
var sid = session_start("Solve");
// ... 通常の send / now / future ...
session_end(sid);
```

違反は `session_events()` に記録される。HM 推論とは独立な動的検査として位置付ける。

3.3 LLM 統合

`ai_call` の API:

```
var r1 = ai_call("prompt"); // env 自動選択 (default: Gemini)
var r2 = ai_call(1, "prompt"); // 1 = Gemini
var r3 = ai_call(2, "prompt"); // 2 = Anthropic (Claude)
var r4 = ai_call(3, "prompt"); // 3 = OpenAI (ChatGPT)
var r5 = ai_call_with_system("sys", "p"); // システムプロンプト付き
var r6 = ai_call_retry(3, "prompt"); // 自動リトライ
```

ランタイムは以下を提供する:

- トークン予算管理 (`ABCL_AI_TOKEN_BUDGET`, `ai_remaining()`).

- 同時実行上限 (ABCL_AI_MAX_CONCURRENT).
- フォールバック連鎖 (ABCL_AI_FALLBACK_MODELS).
- テスト用 mock プロバイダ (ABCL_AI_PROVIDER=mock で API 鍵不要, オフライン smoke 用).

3.4 3 並行モデルの統一

AIPL のセマンティクスは, 以下の異なる並行モデル上で同型に実行できるように設計されている:

1. **1:1 OS スレッド層**: 各アクターが独立した kernel-scheduled なスレッドとして走る. Python の `threading.Thread`, OCaml の `Thread.create`, C の `pthread_create` が該当. ブロッキング I/O が他アクターを止めない代わりに, kernel スケジュールのオーバヘッドにより ~ 1000 アクター程度がスケール上限.
2. **M:N 軽量スレッド層**: ランタイムが小さな OS スレッドプール上でアクターを多重化する. Pony actor / Erlang BEAM process / Go goroutine が該当. $\sim 10^5$ – 10^7 アクターまで実用的にスケール.
3. **協調イベントループ層**: 単一 OS スレッドで `setTimeout(0)` ベースに mailbox を順次処理. `browser-abcl` と `node-aipl-server` が該当. 並行性は無いがメッセージ間のインターリーブは保たれ, ブラウザ DOM に統合できる.

同一の `.abcl` ファイルが, コード変更なくこの三層のいずれにも配置可能であることが本研究の重要な実証成果である.

4 実装

AIPL は単一言語に対して 7 種のランタイム実装と 8 種のコード生成ターゲットを持つ. ランタイムはソースコードをパースしてそのまま解釈実行する. コード生成ターゲットは `aipl2c` コマンドで他言語のソースコードに変換する経路である.

4.1 7 ランタイム

短縮名	ソース	型システム	特徴
Python (注釈)	<code>src/python-aipl/</code>	注釈ベース (gradual)	機能最完備. Phase 11–17 + 動的 compile + メソッドインジェクション
Python (推論)	<code>src/python-aipl-infer/</code>	HM 推論	<code>python-aipl</code> からランタイムを継承, 型検査層のみ差替え
OCaml	<code>src/*.ml</code>	HM 推論	正典実装. <code>web_gateway</code> に WebSocket 内蔵
JS-OCaml	<code>src/app.js</code> 等 + OCaml gateway	(= OCaml)	ブラウザ JS が HTTP で OCaml バックエンドを駆動
JS-Browser	<code>src/browser-abcl/</code>	flow-sensitive	ブラウザ単体, canvas デモ可
JS-Node	<code>src/node-aipl-server/</code>	flow-sensitive	Node HTTP server. <code>/api/typecheck</code> + <code>/ws</code>
C (aipl2c)	<code>src/aipl2c.ml</code> + 各種 C runtime	HM + 特殊化	コード生成経路. SDL2/Xinu/Python 他にも展開

4.2 8 コード生成ターゲット

aipl2c は OCaml で書かれたコード生成器で, フラグにより出力言語を切り替える:

フラグ	ターゲット	アクター単位	用途
(なし)	C + pthread	1:1 OS スレッド	軽量スタンドアロン
(-lSDL2)	C + SDL2 GUI	同上 + GUI スレッド	可視化デモ
--xinu	Xinu C	1:1 OS プロセス	組込み OS
--python	Python	1:1 OS スレッド	配布用ピュア Python
--pony	Pony	M:N (Pony 作業窃取)	capability-secure な研究用
--erlang	Erlang	M:N (BEAM)	軽量プロセス, hot reload
--go	Go	M:N (goroutine)	プロダクション展開
--prolog	SWI-Prolog	1:1 OS スレッド (SWI)	論理プログラミング統合

すべて単一のソースを共有する .Hello.abcl を例に取れば, 8 ターゲットすべてで意味的に同じ出力 (Hello! count = 5 等) を得る.

4.3 コード生成の中核技術

HM 推論結果に基づくコード特殊化 (C ターゲット) src/c_translator.ml は推論済みの型をクラスフィールド・メソッドパラメータ・ローカル変数の C 宣言に直接反映する:

```
// AIPL: class Hello { float count = 0.; method init(n) {...} }
// 生成 C:
static void Hello_init(int self_id, int sender_id,
                      value_t* args, int n_args) {
    long p_n = (n_args > 0)
        ? (args[0].tag == V_INT ? args[0].i : (long)args[0].f)
        : 0L;
    objects[self_id].fields[F_Hello_count] = mk_float((double)p_n);
    // ...
}
```

value_t タグ付き共用体はアクター間メッセージ境界のみで使用し, メソッド本体内部はネイティブ C 型に特殊化される. 算術演算は v_binop() ディスパッチを経由せず直接 (double)a + (double)b を発行するため, ホットパスでのオーバーヘッドが消える.

ターゲット言語のアクター意味論への射影 8 ターゲットそれぞれが, AIPL の send obj.method(args) 構文を異なる仕組みに対応付けている:

ターゲット AIPL send の射影

ターゲット	AIPL send の射影
C+pthread	enqueue(self_id, obj_id, "method", args)
Python	obj._enqueue("method", args)
Pony	obj.method(args) (Pony の actor は async デフォルト)
Erlang	Pid ! {method, Args}
Go	obj.Method(args) 経由で obj.mailbox <- msgT{...}
SWI-Prolog	thread_send_message(Tid, method(Args))

二段階初期化 (Pony / Go / Erlang) AIPL の new C(args) は構築と init 呼び出しを暗黙に合成するが, Pony や Go の strict なコンストラクタ意味論では, 構築時にクロスアクター参照が未確定でも安全に init を起動する必要がある. これを次の二段階パターンで解決した:

```
// AIPL:
var p = new Pinger();
```

```

var q = new Ponger();
send p.bind(q);

// Go 出力 (抜粋):
p := NewPinger()      // 構築のみ, init body はまだ走らない
q := NewPonger()
p.Init()              // _aipl_init behaviour に init body を委譲
q.Init()
p.Bind(q)             // クロスアクター参照が確実に存在する状態で実行

```

5 セルフホスト塔

aipl-self-host/ 配下に, AIPL を AIPL 自身で記述した 9 段階の塔がある. Python ランタイムをホストとして AIPL コードが AIPL コードを評価する構造:

Level	実装内容	何が動くか	Sm.
A	metacircular eval(ast, env)	Arith, Control, Fib, Hello	4/4
B-1	Phase 11 type checker	6 sample	6/6
B-2	Phase 12 capability effects	4 sample	4/4
B-3	Phase 13 channel element types	5 sample	5/5
B-4	Phase 14 linear / use-after-move	4 sample	4/4
B-5	Phase 15 owned / pub visibility	4 sample	4/4
C	lexer + parser + eval パイプライン	AIPL が AIPL を読む	3/3
C-2	actor scheduler + mailbox	6 sample	6/6
C-3	now / Reply 同期送信	1 sample	1/1
合計		全 9 段階	37/37

Phase monotonicity の自動証明. 各 Level B-N の checker は, それ自身の .abcl ソースに対しても自分のルールが矛盾なく適用できることを SampleSelfConsistency.abcl で確認している. これは「Phase N の checker は Phase N の規約に従って書かれている」という不変条件の経験的検証となる.

6 評価

6.1 サンプル網羅性

ランタイムごとに到達可能なサンプル数 (core + AI 統合 + remote-actor):

カテゴリ	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
core	33	33	57	57	5	5	57
AI 統合	11	11	8	8	0	0	8
remote-actor	8	8	1	1	0	0	1
合計	52	52	66	66	5	5	66

クロスカットとして aipl-self-host が 48 個, docker/cross の言語間相互運用サンプルが 4 個ある.

6.2 スモークテスト結果 (2026-05-13 最新)

対象	PASS	xfail	FAIL
OCaml smoke (run_all_smoke_tests.sh)	48	0	9*
browser-abcl (syntax / parse / typecheck)	7+4+4	0	0
node-aipl-server (HTTP + WS)	10	0	0
python-aipl-inferred	20	0	0
Pony codegen	2	1	0
Erlang codegen	2	1	0
Go codegen	2	1	0
Prolog codegen	2	1	0
C + WebSocket (libwebsockets)	1	0	0
aipl-self-host (全 9 Level)	37	0	0

* OCaml smoke の 9 fail は abcl/ の Gui/Py/Xinu サンプル固有の既存型問題 (var partner = 0; を後で actor 参照で上書きするパターン). 本実装の追加変更による回帰ではない.

6.3 型推論のクロス検証

3 つの HM 実装 (OCaml / Python-inferred / C) は, 同一の .abcl 入力に対して同一の推論型を出力することを確認した:

```
// abcl/Hello.abcl
class Hello { float count = 0.; method init(n) { count = n; } method greet() {...} }
var h = new Hello(5);
```

実装	推論結果
OCaml	count:float, init:(int)->unit, greet:()->unit
Python-inferred	count:float, init:(int)->unit, greet:()->unit
C (aipl2c)	count:float, init:(int)->unit, greet:()->unit

3 実装すべてで init の引数型が呼び出しサイト (new Hello(5)) の整数リテラルから int に モノモルフィゼーションされ, count のフィールド型が初期値 0. から float に固定される. HM 推論は実装言語に依存しない普遍的なアルゴリズムであることを, この一致が経験的に示している.

6.4 LLM 統合の実証

ai_call の三送信パターン全てを 5 ランタイム (Python-A, Python-I, OCaml, JS-Browser, JS-Node) で確認した:

パターン	Py-A	Py-I	OCaml	JS-B	JS-N
now	✓	✓	✓	✓	✓
future + await	✓	✓	✓	✓	✓
send + callback	✓	✓	✓	✓	✓

OCaml では検査中に二つのバグを発見・修正した: (a) ABCL_AI_PROVIDER=mock が明示的な provider 引数を上書きすべき箇所での優先順位エラー, (b) 最上位 send a.ask("hello", rcv) が引数を評価しないまま配送する pre-existing バグ.

6.5 WebSocket 統合

全 7 ランタイムで WebSocket サーバを動かせる状態にした:

実装	経路
OCaml	web_gateway.ml に内蔵 (RFC 6455 自前実装,40 LOC)
JS-OCaml	OCaml バックエンド経由
Python (annot/inf)	aipl_websocket.py (websockets ライブラリ)
JS-Browser	ブラウザネイティブ WebSocket API
JS-Node	ws ライブラリ, /ws?sid= + /api/broadcast
C (aipl2c)	abcl_ws_runtime.c (libwebsockets)

ワイヤープロトコルは全実装で統一: ?sid=<group> クエリでブロードキャストグループに参加, 接続時に {"kind":"welcome","sid":"...","peers":N} を受信, 以後同一 sid の peers にメッセージが転送される.

7 関連研究

ABCL/1 [2] 本研究の直接の先祖.AIPL は ABCL/1 の三種送信を継承し, 現代的な型システムと LLM 統合を上重ねた.

Erlang/OTP [8] BEAM 仮想機械上の軽量プロセスとメッセージ受信パターンマッチング. AIPL の --erlang ターゲットはこの語彙で出力するため, 意味論的に最も近い.

Akka [9] Scala の上のアクターライブラリ.tell (= AIPL send) と ask (= AIPL now) の対応がある.AIPL はライブラリではなく言語仕様レベルで同じセマンティクスを提供する.

Pony [10] Reference capabilities (iso / val / ref / box / tag) でアクター間のデータ共有を静的に検査する.AIPL の Phase 14 (linear) と Phase 15 (owned) はこの方向への近似であり, --pony ターゲットは両者の概念的対応を確認する自然な経路となる.

Project Loom (Java) [11] Java 21+ の virtual threads.AIPL の Phase 17 (scope { future ... }) は Loom の StructuredTaskScope と意味論がほぼ同型.

Session types [7] 以降の研究.AIPL のセッション型は完全に動的検査側に置いたため, 複雑なプロトコルでも記述しやすい一方で静的な健全性保証は犠牲になっている. この設計選択は AIPL の研究的立ち位置 (実用主義) を反映している.

8 結論と今後の課題

本稿では, アクター言語 AIPL を 7 ランタイム + 8 コード生成ターゲットの **一言語多実装**として設計・実装した経過を報告した. 主な貢献は:

1. LLM 連携を第一級プリミティブ ai_call として組み込み, プロバイダ・予算・並行性をランタイムが管理する設計.
2. Hindley-Milner 推論を中心とする多層型システム. 実装ごとにどの層を有効にするかが独立に選べる.
3. 同一の .abcl を OS スレッド層・M:N 層・協調イベントループ層の三並行モデルすべてに展開できることの検証.
4. AIPL を AIPL で記述した 9 段階のセルフホスト塔 (aipl-self-host/) で言語仕様の自己一貫性を経験的に示した.

今後の課題

- **JVM 系コード生成** (Kotlin / Scala): Loom の virtual threads を活用すれば,JVM エコシステムへの自然な橋渡しとなる.
- **Swift コード生成**: Swift 5.5+ の actor 言語機能は AIPL の Phase 17 scope と意味論が同型で,Apple エコシステムへの到達範囲が広がる.
- **WebAssembly ターゲット**: 既存の C ターゲットを経由せず直接 WASM を出力すれば,browser / edge / serverless へのデプロイが軽量化される.
- **C ターゲットの機能補完**: 現状 become / select / now の reply slot / ai_call が C ランタイムに未実装.libcurl + 拡張 mailbox レイヤの追加で対応可能.
- **セルフホストの他ターゲットへの展開**: 現在 Python ランタイム上でのみ動く Level A-C-3 を,--pony や --go 出力上でも動かす実験は,セマンティクス保存の強い証拠となる.
- **Phase 18 予測**: aice-evolution-v2 の MAP-Elites GA を Phase 17 状態の AIPL に対して走らせ,次世代軸を経験的に導出する.

References

- [1] Hewitt, C., Bishop, P., and Steiger, R. (1973). *A Universal Modular ACTOR Formalism for Artificial Intelligence*. IJCAI-73.
- [2] Yonezawa, A., Shibayama, E., Takada, T., and Honda, Y. (1986). *Modelling and Programming in an Object-Oriented Concurrent Language ABCL/1*. Object-Oriented Concurrent Programming, MIT Press.
- [3] Hoare, C.A.R. (1978). *Communicating Sequential Processes*. CACM 21(8).
- [4] Lucassen, J.M. and Gifford, D.K. (1988). *Polymorphic Effect Systems*. POPL.
- [5] Wadler, P. (1990). *Linear Types Can Change the World!*. Programming Concepts and Methods.
- [6] Clarke, D. and Drossopoulou, S. (2003). *Ownership, Encapsulation and the Disjointness of Type and Effect*. OOPSLA.
- [7] Honda, K., Vasconcelos, V.T., and Kubo, M. (1998). *Language Primitives and Type Discipline for Structured Communication-Based Programming*. ESOP.
- [8] Armstrong, J. (2007). *Programming Erlang: Software for a Concurrent World*. Pragmatic Bookshelf.
- [9] Lightbend (2010–). *Akka Documentation*. <https://akka.io/docs/>.
- [10] Clebsch, S., Drossopoulou, S., Blessing, S., and McNeil, A. (2015). *Deny Capabilities for Safe, Fast Actors*. AGERE!.
- [11] Pressler, R. (2023). *JEP 444: Virtual Threads*. OpenJDK. <https://openjdk.org/jeps/444>.

本稿で記述した実装は <https://github.com/yaskodama/aio-cs-claude> で公開されている. スモークテストの最新結果と機能比較表は同リポジトリの docs/AIPL_Runtime_Feature_Matrix.{md,tex,pdf} に随時更新される.